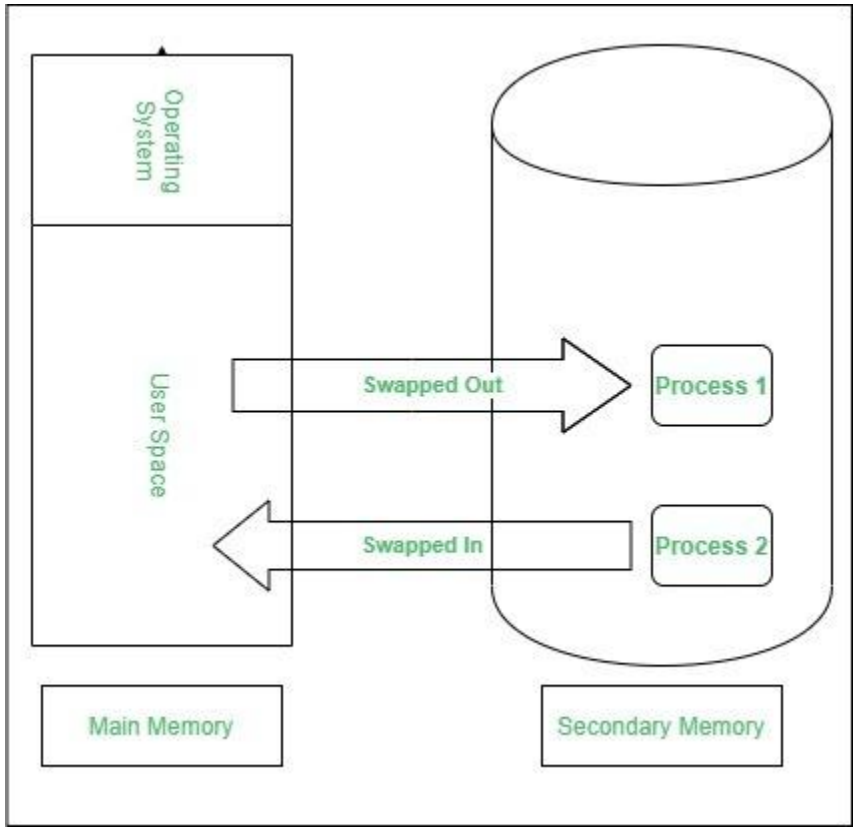
Memory Management Contd2

Dr. Jit Mukherjee

Swapping

- A process must be in memory to be executed
  - *Backing store* - Swapped temporarily out
    - A fast disk
  - Dispatcher checks in memory?
    - No free memory -> swap
  - Round Robin - After time slice
- A variant of swapping -> Priority Scheduling -> *roll out, roll in*
- Swapped back into same location
  - Address binding in assembly or load time
  - Execution time binding -> flexible
- Cost -> high -> Major part Transfer Time



Swapping Cost

- user process - 10 mb, backing store transfer rate 40 mb/sec
  - Assuming no head seek and average latency 8 millisecond
  - Swap time -> 250 + 8 milisec. Swap in and out -> 516 milisec
  - Time quantum >> 0.516 second
- Total Transfer Time ~ Amount of Memory Swapped
- 512MB main memory, OS -> 25MB => For user -> 487MB
  - 10MB -> 258 millisecond and 256MB -> 6.4 seconds
  - *Swap only what is used* -> request memory, release memory
- Constraint -> In between I/O, I/O is queued
  - Never swap a process pending I/O
- Modern system - swapping is disabled until many process are running and cross the threshold of memory
  - Halted when load is reduced
  - Old system - User select the swap out process to run again

Contiguous Memory Allocation

- Main memory -> Operating system + processes -> efficient way possible
- Typically divided into two part
  - Low memory - Generally OS
  - High Memory - Processes
- Several processes reside in memory at the same time.
- Contiguous memory allocation - each process is contained ina single contiguous section of memory
- Relocation register + Limit Register
  - MMU - Checks and relocates
- Dispatcher loads the relocation and limit register with correct value -> Context Switch -> every address is checked -> Memory Protection
- Transient OS -> Change the size of the operating system

Memory Allocation

- Typical - Divide the memory into several fixed sized partitions
  - One partition - one process
  - Degree of multiprogramming -> number of partitions
- Multiple partition method -> a partition is free, a process from input queue
  - A process terminate -> partition available for another
  - IBM OS, No longer in use. Batch environment
- OS keeps a table which parts of the memory available and allocated
- Initially all the memory available -> A large block of available memory -> hole
  - Find a hole large enough to accommodate the process
  - Allocate as much memory needed and keep the rest for future use
- A process enters a system -> input queue
  - OS checks memory requirement and available space
  - A process is allocated to a space -> Load into memory -> it can compete for CPU
  - Terminates -> Release -> Next Process

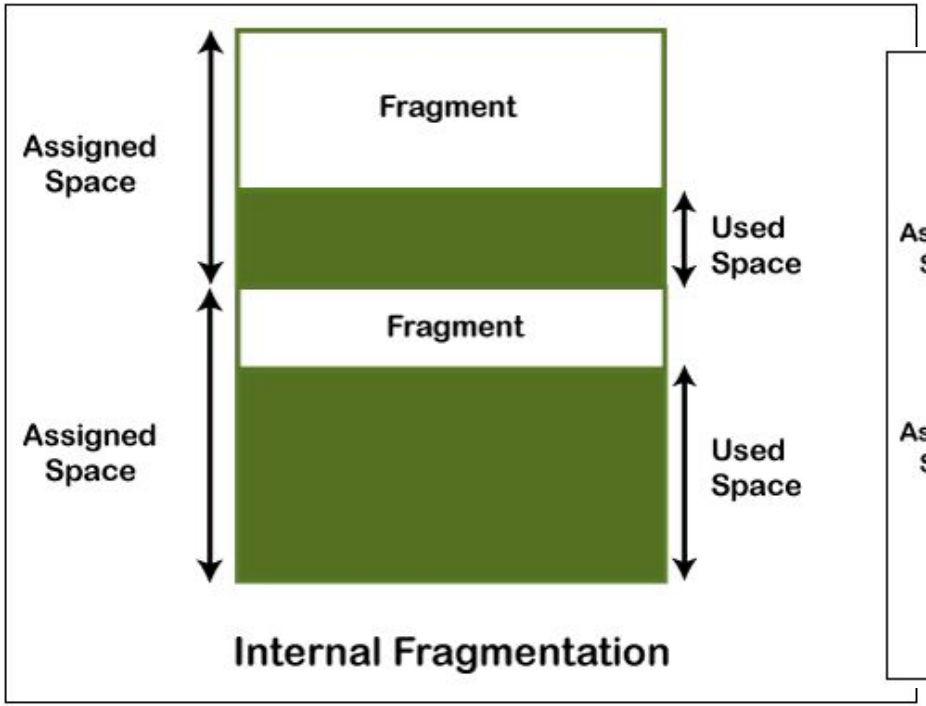
Memory Allocation

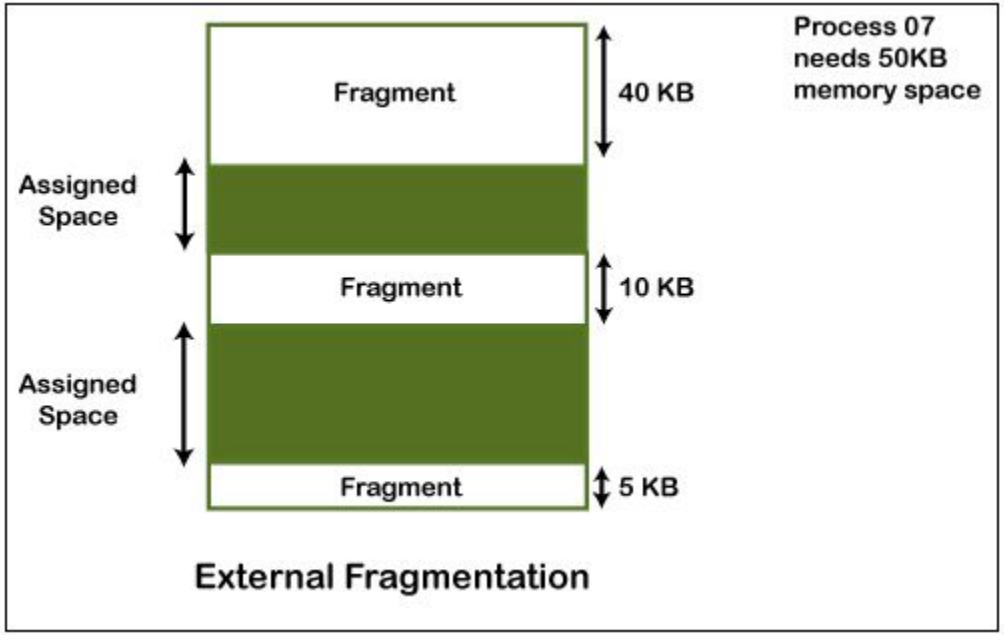
- At t_0 -> List of available block sizes and input queue
  - Input queue to a scheduling algorithm
- Memory is allocated until it can not be further satisfied
  - No hole is large enough to hold a process
  - Wait for large block or skip down the input queue for smaller process
- At t_k -> Various sizes of holes scattered over a memory
  - Searches for hole that is large enough for the process
- Hole is too large -> Split it
  - One part have the process and the other one back to the available list
- Process terminates and releases the hole
  - New hole adjacent to each other -> make a bigger one
  - Check the demand and allocation of the process
- *General Dynamic storage allocation -*
  - First Fit
  - Best Fit
  - Worst Fit

Fragmentation

- Both first fit and best fit suffers from external fragmentation
- Process loaded and removed -> Free memory space -> isolated little spaces -> External fragmentation exists when there are enough total available memory spaces but not contiguous, hence does not satisfy -> Severe problem
- Another factor -> which end of a free block is allocated
- First fit -> for N allocated blocks 0.5N blocks will be lost to external fragmentation -> *50-percent rule*
- *Compaction* -> shuffle the memory content into one
  - Problem if relocation is static
  - Cost of Compaction. Move all process to one end an holes to another end
- Permit logical address space to noncontiguous
- Internal fragmentation -> hole of 18,464 byte, process request 18,462 byte, 2 byte lost to internal fragmentation

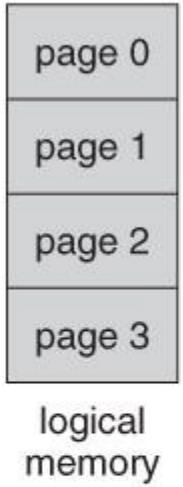
Fragmentation

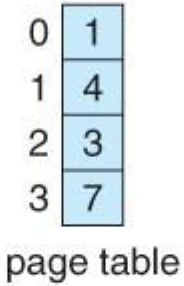


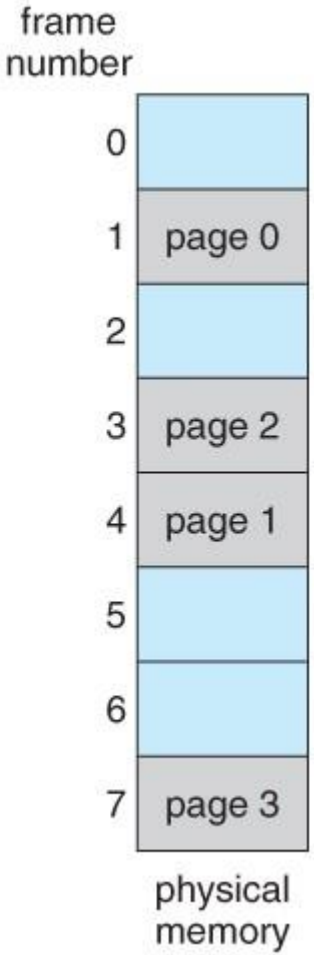


Introduction to Paging

- Permits physical address space to be noncontiguous.
- Physical memory breaks into fixed sized blocks -> frames
- Logical memory breaks into fixed sized blocks -> pages
- Every address -> page number (p) and a page offset (d)
- Page table -> page number is the index
  - Contains base address of each page in physical memory







Basic Method and Hardware Support

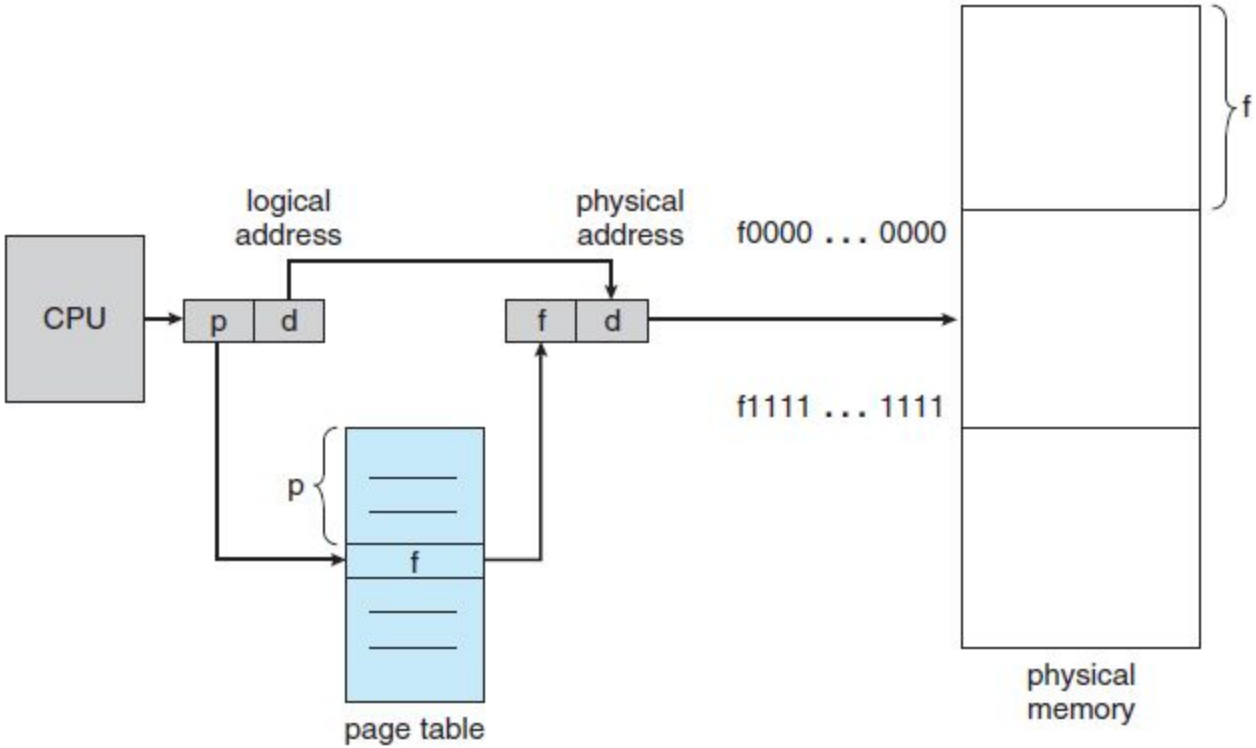


Figure 8.10 Paging hardware.